

SISTEMAS DISTRIBUÍDOS EXAMEN FINAL

PROYECTO FINAL

OBJETIVO: Desplegar el proyecto final de curso, evaluando conocimientos técnicos en manejo de sesiones, HTTP, WS, persistencia y replicación.

Todo lo de los talleres anteriores se mantiene:

- Servidor autoritativo. El cliente no calcula posiciones.
- Sin Redis, sin Kafka, sin librerías de consenso. Todo con ws, express, node:sqlite.
- JWT con secret compartida. Sin sesiones en memoria del lado del auth.
- Sin proxy reverso delante del sistema. El cliente conoce la lista de servidores.
- Sin sticky sessions.

Si quieres violar alguna de estas restricciones por una razón concreta (ej: agregaste un proxy porque tu extensión lo requiere), lo justificas en el README.

Para el proyecto final se piden tres cosas, en orden de importancia:

1. Integración limpia de los 4 talleres en un solo repositorio coherente que funcione tanto en local como en despliegue a través de un túnel.
2. Una extensión elegida de la lista de tracks (ver más adelante). Debe estar completamente integrada con el sistema existente. Si eliges dos tracks pequeños en lugar de uno grande, lo justificas en el README.
3. El círculo que se mueve en un canvas es la base mínima. Acá deciden qué juego es: un Haxball (fútbol), un Agar.io (comer puntos y crecer), un Slither (gusanito), un "atrapadas", una arena de combate con barras de vida, lo que se les ocurra. La adaptación debe ser profunda, no cosmética.
4. README técnico de calidad profesional que documente arquitectura, decisiones, modos de falla conocidos, y cómo levantar todo. No se trata de un "instructions.txt", se trata de un documento que un ingeniero externo pueda leer y entender el sistema en menos de 10 minutos.

1. Gameplay sencillo

Lo que cambia entre grupos es qué juego eligen ser.

Qué debe tener tu juego

- Una mecánica de juego no trivial que va más allá de "moverse en un canvas vacío". Por ejemplo:
 - Una pelota física que rebota y se puede empujar (Haxball).
 - Puntos en el mapa que aparecen y al tocarlos creces / sumas puntos (Agar.io).
 - Tu cuerpo es un trail que crece, chocar con otro trail te mata (Slither).
 - Una zona segura que se achica con el tiempo, queda el último vivo (Battle royale).
 - Equipos, arcos, goles, marcador (cualquier deporte virtual).
 - Cualquier otra que se les ocurra. Piensen en simplicidad, no van a hacer el siguiente GOTY en una semana.
- Estado de juego adicional al de movimiento que se replica entre coordinadores. No basta con que cambie el color del círculo, debe haber algo de gameplay nuevo que el servidor mantenga autoritativamente.
- Condiciones de victoria, fin de partida, o progresión. Algo que distinga "estar jugando" de "estar moviendo un círculo".

Qué NO cuenta como adaptación

- Cambiar el color de fondo, agregar una imagen de logo, poner una skin distinta a los círculos.
- Sumar un chat sin más (eso es un feature, no es una mecánica de juego).
- Mostrar más estadísticas en la UI.
- "Hicimos los círculos más rápidos."

Cómo se evalúa

La adaptación se evalúa por integración con el sistema y profundidad de la mecánica, no por valor de producción. Un Haxball feo donde la pelota se replica correctamente entre coordinadores vale mucho más que un Agar.io con animaciones espectaculares donde el estado de las comidas es local al cliente.

2. Lista de tracks

Elige dos: uno principal y uno secundario.

- Principal: La demo obligatoria del track debe funcionar completa. Manejo de casos límite. Profundidad técnica.
- Secundario: Versión más acotada. La demo obligatoria funciona, pero puedes tomar atajos razonables (documentados en el README).

Si tu propuesta no encaja en ninguno, puedes proponer un track propio, pero debe ser aprobado por el docente antes de empezar.

2.1. Persistencia del estado del juego

Problema: Hoy si matas todos los coordinadores a la vez, todos los jugadores aparecen en posiciones aleatorias de spawn cuando vuelven. El estado del juego es efímero.

Qué implementas:

- Cada coordinador persiste el estado del mundo (posiciones, extras) cada N segundos a su propia SQLite.
- Cuando un coordinador arranca, recupera el estado más reciente.
- Cuando un coordinador entra al mesh, los demás le envían su snapshot y reconcilia con el suyo si tenía algo persistido (el más reciente gana).

Lo difícil: ¿Qué haces si dos coordinadores recuperan un estado distinto del mismo jugador? ¿Cuál gana? ¿Y si ambos tienen estado más nuevo que la realidad porque el otro coordinador murió hace 5 minutos?

Demo obligatoria: Matas los 3 coordinadores con jugadores en posiciones específicas (ej: alice en x=400). Reinicias los 3. Alice aparece de nuevo en x≈400, no en una posición aleatoria.

2.2. Auditoría y observabilidad del sistema

Problema: Hoy no tienes forma de ver qué está pasando en el sistema sin parsear logs a mano. Si algo se rompe en producción, estás ciego.

Qué implementas:

- Endpoint `/metrics` en cada servicio (auth y coordinador) que devuelve, en JSON o formato Prometheus, mínimo: requests/segundo, latencia p50/p95/p99, conexiones WS activas, tick rate efectivo, jugadores locales y remotos, replicación lag (en el coord: cuánto demora un `intent_replicate` en propagarse).
- Un dashboard HTML simple (servido desde cualquier auth o un servicio nuevo) que pollea los `/metrics` cada 1-2s y muestra el estado del cluster en vivo: quién es líder, cuántos jugadores están en cada coord, latencia entre nodos.
- Trazas mínimas: cada request HTTP y cada mensaje WS importante lleva un `traceld` que se propaga. Cuando algo falla, podés reconstruir qué pasó.

Lo difícil: Medir latencia de replicación entre coordinadores requiere que mandes el timestamp en el mensaje y lo midas al otro lado, sabiendo que los relojes pueden estar desfasados. Vas a tener que pensar en eso.

Demo obligatoria: Durante la sustentación, tienes el dashboard abierto. Causas un evento (matás un coord, registras un usuario, etc.) y el dashboard lo refleja en menos de 3 segundos. Muestras las trazas de una request específica atravesando el sistema.

2.3. Seguridad y robustez

Problema: Hoy alguien con DevTools puede inundar al coordinador con mensajes o intentar password spray al auth, y no pasa nada hasta que algo se rompe.

Qué implementas:

- Rate limiting en el auth (intentos de login por IP/username, registros por IP). Ataques de fuerza bruta deben ser detectables y mitigables.
- Rate limiting en el coordinador para mensajes WS (intents, extras_update). Un cliente que mande 1000 mensajes/segundo debe ser desconectado o silenciado.
- Refresh tokens: los JWT de 1 hora son frágiles. Implementa un mecanismo de renovación que NO requiera re-login. Esto fuerza pensar en qué pasa cuando el refresh token llega a una réplica de auth, etc.
- Logging de auditoría: cada evento de seguridad (login fallido, token rechazado, rate limit hit) debe quedar registrado con timestamp, IP, usuario.

Lo difícil: El refresh token requiere persistirlo (a diferencia del access token), y eso implica replicación entre auths. ¿Cómo lo haces sin romper la arquitectura existente?

Demo obligatoria: Simulas un ataque de fuerza bruta (script que prueba 100 passwords contra un usuario) en vivo. Mostrás que después del intento N, el sistema rechaza ese username/IP por un tiempo. Mostrás los logs de auditoría.

2.4. Particionamiento del mundo (sharding)

Problema: Hoy todos los coordinadores tienen el estado de todos los jugadores. Si tu juego escala a 10.000 jugadores, cada coordinador tiene 10.000 posiciones aunque solo le sirva a 100. Eso no escala.

Qué implementas:

- El mundo se divide en regiones espaciales (ej: 4 cuadrantes). Cada región la "lleva" un coordinador (su homeCoord para esa región).
- Un jugador físicamente está en una región. Si se mueve a otra región, migra al coordinador correspondiente.
- Los coordinadores solo replican el estado de los jugadores en regiones que les interesan (su propia región y las adyacentes, porque los jugadores cerca del borde necesitan ver al otro lado).

Lo difícil: La migración. Cuando un jugador cruza el borde, el WS sigue conectado a un coord pero el "dueño" cambia. Hay que decidir si el WS se redirige (cliente reconecta) o el coord original sigue actuando como proxy.

Demo obligatoria: Tienes un mundo con 4 regiones y 3 coordinadores activos. Muestras un jugador moviéndose entre regiones y muestras en `/status` cómo cambia su `homeCoord`. Mostrás que un jugador en la región noroeste no recibe el estado de un jugador en la región sureste (lejos), pero sí recibe el estado de uno en la región noreste (adyacente).

2.5. Spectator mode

Problema: Hoy solo los jugadores logueados ven el juego. No hay forma de observar sin participar.

Qué implementas:

- Endpoint en el coordinador que acepta conexiones WS sin autenticación como "spectator". Reciben el `state` igual que un jugador pero no pueden enviar `intent` ni `extras_update`.
- Una vista de spectator en el cliente: pantalla parecida a `game.html` pero sin controles, con un indicador claro de "MODO ESPECTADOR".
- El coordinador distingue entre spectators y jugadores en su `connectedPlayers` (los spectators no cuentan para load balancing, o cuentan menos).
- Validación del lado del servidor: si un spectator intenta mandar `intent`, lo ignoras sin crashear.

Lo difícil: Los spectators no consumen recursos despreciables. Un coord con 100 jugadores y 1000 spectators tiene que serializar y enviar state a los 1100. ¿Cómo lo manejas? ¿Limitas cuántos spectators por coord? ¿Hay un coord especial "de spectators"? Justifica.

Demo obligatoria: Abres una pestaña como spectator (sin login) y otra como jugador. El spectator ve al jugador moverse pero no tiene controles. Si fuerzas un `intent` desde DevTools del spectator, el servidor lo ignora.

2.6. Chat distribuido

Problema: Hoy los jugadores no tienen forma de comunicarse durante el juego.

Qué implementas:

- Mensajes de chat enviados por WS al coordinador local. El coordinador los replica a los otros coords vía mesh.
- Validación: longitud máxima, rate limiting.
- Persistencia mínima: cada coord mantiene los últimos M mensajes en memoria. Un cliente recién conectado los recibe.
- Filtrado básico: no permitir caracteres de control, escapar HTML en el cliente para evitar XSS.

Lo difícil: El orden de los mensajes. Si Alice en coord-A escribe "hola" y Bob en coord-B escribe "qué tal" a los pocos milisegundos, ¿en qué orden los ve Charlie en coord-C?

Demo obligatoria: Tres clientes en tres coords distintos mandan mensajes. Todos los ven en un orden coherente. Muestras que el rate limiting funciona.

3. Entrega y sustentación

Qué se entrega:

1. **Link al repo de GitHub.** Commits incrementales, no uno solo.
2. **README completo.**
3. **Sistema corriendo con túneles en ngrok durante la sustentación.**

Qué se sustenta:

1. Demo del sistema completo: El grupo lo muestra. Esperan que el docente pida las demos críticas (login Google, autoridad del servidor, fallover de coord, fallover de auth) y las demos de los dos tracks. (5 min).
2. Demo específica del gameplay adaptado. Explican qué es el juego y cómo lo implementaron en el modelo autoritativo. (3 min).
3. Cada integrante explica una pieza distinta del sistema. El docente elige quién explica qué. (5 min).
4. Preguntas técnicas. (4 min).
5. Cierre: qué harían distinto, qué limitaciones quedaron. (3 min).